

TLS-Fingerprint Scoping of Application-Layer DDoS Mitigation: Calibration Floors and Collateral on a CDN Operator’s Endpoints

Natanael Oliveira, Arthur Kobielski, Marcos Luna, Luigi Carro, Jéferson Campos Nobre, and Eder John Scheid
Institute of Informatics, Universidade Federal do Rio Grande do Sul (UFRGS), Porto Alegre, Brazil
{natanael.oliveira, arthur.kobielski, marcos.luna, carro, jcnobre, ejscheid}@inf.ufrgs.br

Abstract—A low-rate distributed application-layer flood keeps every source below per-source thresholds, so the decision is which clients to block. A binomial test, calibrated on attack-free windows, names the TLS fingerprints over-represented among an alarm’s origins against the endpoint’s profile. On generated traffic the alarm’s most common fingerprint blocks no attacker and 39% of legitimate traffic from five TLS stacks on, while profile-relative scopes block 90% of a botnet on up to 25 stacks absent from legitimate traffic, with no observed collateral. On a CDN’s four endpoints, the binomial configuration misfires on 0.1% of clean windows of the days it was chosen on, blocking a median third of their clients. A held-out day, on which its gate barely opened, is consistent but weakly informative. Client fleets set a calibration floor: a 25-stack botnet on rare fingerprints real clients share is named only past 8% of the busiest endpoint’s window and 4 to 19 median windows elsewhere, and never on the 4 to 28 most common. On the busiest endpoint the distinct-origin gate lets the configuration stop only 12% of a botnet a tenth the window’s size. Post hoc, the unseen-fingerprint filter as its own trigger stops 90% of it there on new stacks within the budget, and a cross-fitted beta-binomial behind a seasonal gate 15% on shared stacks but 20% on new ones. WAF verdicts cannot serve as labels. An OWL ontology specifies the counts and the scope, a JA4 filter no standard we examined expresses in its core vocabulary.

Index Terms—security management, application-layer DDoS, mitigation policy, TLS fingerprinting, statistical calibration, ontology

I. INTRODUCTION

Application-layer Distributed Denial of Service (DDoS) attacks span two extremes. At one end, HTTP/2 Rapid Reset (CVE-2023-44487) pushed mitigation peaks to 398 million requests per second [1], a volume that capacity and scrubbing absorb. At the other end sit low-rate distributed floods, the target of this paper. *Distributed Slow HTTP DoS* and rate-based variants that still hold connections open, such as HULK and GoldenEye [2], exhaust a server from many coordinated origins, each sending requests small enough that no per-source threshold is crossed.

A stealthy campaign also shapes each session to look legitimate on the per-session signals detectors rely on, such as connection duration and route entropy [3]. What remains discriminative lives in the structure *between* sessions: the same

client fingerprint recurring across many origins that converge on one endpoint. Acting on it is a management decision: *which* clients to block, on *what* evidence, with a filter narrow enough to spare the legitimate users of the attacked service. The operator also needs to know, before an attack, where such a filter cannot be built, so that it can fall back there to a rate limit or a challenge.

Existing work addresses detection and leaves that decision open: 47% of 75 application-layer DDoS detectors surveyed derive their features from HTTP sessions, flattened into a numeric vector [4]. Graph-based detectors keep relations between network nodes and stop at a report [5]. Aggregate-based mitigations filter on network and transport headers [6], [7], source-based filtering weighs each blocked prefix against the legitimate traffic it carries [8], and the standard channels that carry a mitigation request filter on the same fields [9], [10]. No published method scopes the block by application-client fingerprint while pricing each such filter in legitimate clients and saying where it stops being possible.

This paper treats the scope of the mitigation as a calibrated statistical decision (Fig. 1). Per endpoint and five-minute window, a volume trigger raises the alarm. The scope is the set of TLS fingerprints over-represented among the alarm’s origins relative to a profile of the endpoint’s normal traffic, at a level set to an empirical quantile of attack-free windows. We target botnets of compromised devices, whose TLS stacks their operator cannot cheaply replace, and evaluate bots that present common fingerprints as the method’s boundary. This paper makes three contributions. (i) On generated traffic we show when the natural scope, the alarm’s most common fingerprint, blocks users and no attackers, and that profile-relative scopes block the attack without labels, stacks that real clients share included, there at a collateral of 2.2%. (ii) On eight days of four endpoints of a CDN operator we measure what calibration costs: the false alarms, the calibration floor at which the scope names a typical botnet stack, and a harder limit, set by the enrichment ratio, past which it names none. We also measure how much of an injected botnet the trigger lets through, compare post hoc triggers that recover part of it, and analyze a held-out day as fixed in advance. (iii) No

TABLE I
COMPARISON WITH RELATED WORK.

Approach	Filter matches	Chosen by	Cost to legitimate users
Aggregate rate control [6], [7]	Header-defined aggregate	Congestion or aggregate score	Rate-limited with the aggregate
Blackholing [12]	Destination prefix, or header fields	The attacked network	All its traffic, or less with finer filters
Source-based filtering [8]	Source prefixes	Optimization under a filter budget	Traffic of the filtered prefixes
Anomaly extraction [13], [14]	Flow feature values	Over-representation in anomalous traffic	None (diagnosis only)
Signature generation [15]	Payload tokens	Coverage against a normal pool	False positives on normal traffic
Puzzles [16]	Every client under attack	The cost a client pays	A puzzle each
DOTS, Flowspec [9], [10]	Network and transport fields	The requesting party	That of the filter carried
L7 detectors [3], [5], [17]	None (verdict only)	Profile or learned model	Detection FPR only
Bot management [18]	JA4	Undisclosed	Undisclosed
This work	TLS fingerprints (JA4)	Calibrated enrichment against the profile	Measured per alarm and window

standard we examined expresses the resulting scope, a filter over JA4 fingerprints, in its core vocabulary. STIX 2.1 holds a JA4 only in a custom extension, OCSF records it but defines no filter [11], and DOTS and Flowspec match only network and transport fields. The counts the decision reads and the scope it exports are specified in an OWL ontology, from which the log store’s query is compiled.

II. RELATED WORK

Table I compares prior scopes and detectors on what a filter matches, how it is chosen and what it costs legitimate users.

Scoping a mitigation. Aggregate-based congestion control identifies the aggregate of an attack by header fields and rate-limits it upstream [6], and a recent form scores aggregates in programmable switches [7]. Blackholing drops all traffic to the attacked prefix, and advanced blackholing narrows it to header fields to limit that collateral damage [12]. Source-based filtering chooses which prefixes to block under a filter budget, trading the attack traffic stopped against the legitimate traffic of those prefixes [8]. Feature distributions detect and classify traffic anomalies [19], and anomaly extraction isolates the feature values over-represented in anomalous traffic [13], [14], the question our test asks of fingerprints, and signature generation checks its candidates against a pool of normal traffic [15]. Defenses against attacks that mimic flash crowds make every client solve a puzzle [16], or schedule each session by its deviation from legitimate workload profiles, weighing the delay imposed on legitimate users [20]. All of them work on network, flow, session or payload features, without the fingerprints of an application’s clients. The IETF DOTS data channel installs filters on network and transport header fields [9], as BGP Flow Specification does [10], and neither can express an application-layer client fingerprint such as JA4.

Layer-7 detection. Layer-7 HTTP DDoS detectors [4] either profile traffic, applying Principal Component Analysis (PCA) to flow statistics [21] or to a per-session behavioral matrix [17], or learn from session features [3]. None retains a relation between sessions. Flash crowds are told apart from floods by how requests spread over a site’s documents [22].

Knowledge graphs and fingerprints. Security knowledge graphs link entities extracted from threat reports, or map countermeasures to ATT&CK as D3FEND does in OWL [23], and how to apply them in operations remains unclear [24].

Graphs built from traffic feed learned detectors: KLAGe [5], the closest to our setting, classifies the nodes of graphs built from network logs. Passive TLS fingerprinting identifies client software from its *ClientHello* [25], and one fingerprint is often shared by the many applications built on one library [26], which is what makes a stack a scope and what limits it. Most bots that claim a browser’s identity present the TLS and header fingerprints of another tool [27]. Industrial bot-management platforms aggregate signals per JA4 over recent traffic [18], with no public model, benchmark or collateral metric.

III. CALIBRATED FINGERPRINT SCOPING

This section states the threat model, the trigger, the scope and its calibration, the floor that calibration imposes, and what the ontology specifies (Fig. 1).

A. Threat Model

The defender operates the attacked application and observes, per request, the TLS *ClientHello*, source address and prefix/ASN, target endpoint, arrival time, and whatever identity material the application exposes. The *ClientHello* gives a JA4 fingerprint [28], a string that summarizes its TLS version, cipher suites, extensions and ALPN. It sees neither the botnet’s control channel nor which sources are coordinated, but can profile the service’s own traffic outside attack episodes. The attacker controls K compromised devices and may shape each session to be statistically indistinguishable from a legitimate one, jittering, rotating or randomizing timing, identities and payloads, the stealth assumption of our evaluation. The damage is exhausted capacity: each session holds a connection and a worker as a legitimate one does, so no per-session rate rises. We also assume the TLS stack is harder to change: a bot that speaks TLS does so through the library its device or toolkit ships, and botnets of compromised devices span many router, camera and DVR models [29]. Presenting a browser’s *ClientHello* takes an impersonation library kept in step with browser releases [30]. Bots that do so, such as headless-browser farms or bots built on TLS-impersonation libraries, fall outside this model, and Section V evaluates them as its boundary.

B. The Trigger

Per endpoint e and five-minute window, let S be the sessions to e and n their distinct origins. The trigger fires

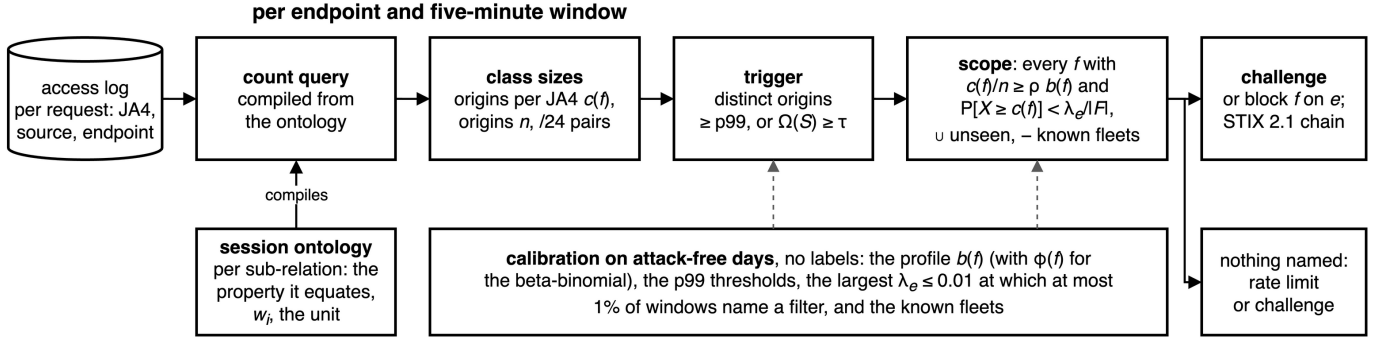


Fig. 1. Calibrated fingerprint scoping. The query compiled from the session ontology returns, per endpoint and window, the class sizes the trigger and the scope read, and every threshold and level comes from attack-free days.

when $n \geq k_{\min} = 5$ and n reaches its 99th percentile over attack-free windows: the **distinct-origin gate** of the production evaluation. The base rule, the protocol’s reference, gates instead on the coordination mass $\Omega(S)$ of the ontology’s rule `CoordinatedHTTPFlood`, with its threshold τ_{cluster} at the same percentile. It is a weighted count of the pairs of distinct origins whose sessions share a JA4, an endpoint or a /24 prefix (Appendix A). Ω is mostly volume: its endpoint term, $0.6\binom{n}{2}$, carries 91% of it in the cluster of a canonical campaign (Appendix D), and the discriminator comes from the scope.

C. The Scope and Its Calibration

How the scope is chosen decides whether the mitigation helps or harms, and the frequency-based choice fails: the alarm’s modal fingerprint selects whatever is common, and on a service under attack that is the legitimate population. A botnet of A attackers spread uniformly over M stacks puts about $0.9A/M$ on each and the remaining tenth on one-off fingerprints, while the benign head holds a share p_1 of the n_b legitimate origins. The modal value is an attack stack only while $0.9A/M > p_1 n_b$: with the measured $p_1 = 38.4\%$ of requests (Section IV), a proxy for origins, and a campaign as large as the legitimate traffic, only while $M < 2.3$. With more stacks the filter blocks users and no attackers.

We therefore scope against a **profile** of the endpoint’s normal traffic, kept by the operator outside attack episodes. Let $b(f)$ be the share of fingerprint f , plus $1/N$, among the profile’s N origin–fingerprint pairs, $c(f)$ the number of the alarm’s origins presenting f , and $n = \sum_f c(f)$. The **binomial enrichment test** admits every f that is enriched, $c(f)/n \geq \rho b(f)$ with $\rho = 3$, and improbable under the background, $\Pr[X \geq c(f)] < \lambda_e/|F|$ for $X \sim \text{Bin}(n, b(f))$, where the Bonferroni divisor $|F|$ counts the fingerprints of profile and alarm. The result, a *set* of fingerprints, covers a fragmented botnet. A per-fingerprint ***z-score***, $z(f) = (c(f) - nb(f))/\sqrt{nb(f)(1-b(f))}$, gives a simpler profile-relative scope. A filter of fingerprints absent from the profile and seen in at least $k_{\min}$ origins (**unseen**) needs no profile statistics, and joined with the test (the **union**) it recovers new stacks too small for the level.

On generated traffic each origin carries one session, and the level stays at 0.01. On production traffic, fleets of legitimate clients switch on together and break the binomial’s independence. The level λ_e is therefore set per endpoint e to an empirical quantile: the largest value up to 0.01 at which the scope names a filter in at most 1% of the endpoint’s attack-free calibration windows, about 2.9 of a day’s 288. At levels such as 10^{-60} the binomial tail is no longer the probability of the data under the model. It is a score, whose threshold that quantile sets. The *z-score*’s threshold is set to the same budget, as the 99th percentile of each calibration window’s largest z , never below 3. Both budgets hold on the calibration windows by construction, and Section V-B measures how far they hold on the days after them.

The profile is what a defender observes in normal operation, so no attack label is involved. It must come from outside the attack episode, because the campaign spans the whole window and would otherwise give every candidate an enrichment of one. An adversary hiding inside a popular benign fingerprint is therefore not enriched, and the test names no scope, the correct report when no discriminator exists.

D. The Calibration Floor

A calibrated level has a price, the **calibration floor**. Among n origins the test names a stack of c only if $c/n \geq \rho b(f)$ and $\Pr[X \geq c] < \lambda_e/|F|$. The floor is the smallest botnet of A attackers on M stacks whose mean stack, $0.9A/M$ attackers in a window of $n_0 + A$ origins with n_0 the median window of the calibration days, passes both, and it grows as the level falls. It marks where the scope names a typical stack. Joined with the test, the unseen filter names a stack absent from the profile once it holds $k_{\min}$ origins, from $\lceil k_{\min} M/0.9 \rceil$ attackers whatever the level, 139 for $M = 25$. The level’s floor thus binds on stacks that real clients also present, and there the ratio sets a harder limit. A stack’s expected share of the window stays below $0.9/M$, so a botnet on M stacks is not expected to enrich a fingerprint of prevalence above $0.9/(\rho M)$, 1.2% for $M = 25$ and $\rho = 3$.

We evaluate two remedies. The first exempts the endpoint’s *known fleets*, fingerprints named at the nominal level in at least 5% of calibration windows, from the scope and from

the calibration. The exemption is an allow-list, on the busiest endpoint of fingerprints carrying about 7% of its profile: a bot presenting a fleet’s fingerprint is not scoped. The second, built after the held-out day was read (Section V-D), is a **beta-binomial background**. Its intra-window correlation $\phi(f)$ is a moment estimate per profile fingerprint over the calibration windows, from $\text{Var}[c(f)] = nb(1-b)(1+(n-1)\phi)$, clipped to $[0, 0.99]$, and each count is tested against a beta-binomial of the same mean. A fingerprint absent from the profile keeps the binomial.

E. The Session Ontology as Specification

The ontology is the specification the operator deploys. Its central class, `ApplicationSession`, links a session to its Identity, source IPAddress, Endpoint and Mitigation, whose scope the evidence chain carries. Sub-properties of a symmetric `relatedTo` type the evidence that links two sessions (Appendix A). The three exercised here are equalities: exact JA4, endpoint and /24 prefix partition the sessions into classes. The ontology annotates each with the session property it equates and `relatedTo` with the unit it counts in. From these annotations we compile the SQL query that returns, per window and endpoint, the class sizes the gate, the base rule and the scope read, given only a binding of properties to columns. No reasoner runs in the evaluation. A new equality signal reaches the compiled query as four triples, plus a binding for any new session property, with no change to any code. When the rule fires, the evidence chain (the rule, the instances, the decomposition of $\Omega(S)$ per sub-relation and the scope) is exported as JSON-LD (Listing 1) and as STIX 2.1 [31], an *indicator* plus a *course-of-action* linked by *mitigates*.

IV. EVALUATION METHODOLOGY

Each traffic source answers one question. A generator calibrated on benign CICIDS2017 sessions and on JA4 measured at a CDN carries the controlled campaigns. Laboratory captures test conventional attacks, with an uncontrolled comparison to the published result of KLAGE, the closest system (Appendix C), and eight days of the CDN’s traffic test the scopes’ false alarms and, with a botnet injected, their detection.

A. Scenarios and Datasets

The evaluation is parameterized by the *degree of distribution* K of the campaign, the number of devices the attacker controls. We report $K = 50$, a small botnet whose origins each stay inside their own rate limit, and $K = 1000$, spread over hundreds of ASNs and prefixes, with negligible per-session signal.

Controlled evaluation: calibrated synthetic traffic. A generator produces distributed Slow HTTP DoS campaigns against one endpoint, with known coordination ground truth, parameterized by K , TLS and identity sharing, IP/ASN dispersion and the time window. Benign fingerprint popularity is shaped as a Zipf curve of exponent α , since real populations are concentrated. We calibrated that curve against 6.33M TLS requests logged at a production edge point of presence

of Azion Technologies¹: 495 distinct fingerprints, the most common accounting for 38.4% of requests and the top ten for 93.8%. The measured curve is bracketed by $\alpha = 1.5$ and $\alpha = 2.0$, and we take $\alpha = 1.5$ as canonical. No published measurement fixes botnet composition, so we sweep it: 90% of the campaign spreads uniformly over M stacks, the conservative choice for a given M , the rest over one-off fingerprints, and $M = 25$ is canonical. A *shared* mode draws the stacks from the profile’s tail, fingerprints real clients also present, and an *adversarial* mode lets the botnet adopt the most common benign fingerprints, as an impersonating bot would.

Legitimate request counts and durations are sampled from the benign sessions of CICIDS2017, all 322 658 for the first and the 40 351 with two or more requests for the second, so Kolmogorov–Smirnov distances of $D = 0.002$ and 0.003 between generated and real sessions only check the sampler. A *stealth* mode draws each attacker session’s attributes from the same distributions, so the campaign reveals itself only through cross-session structure. Attackers start within the first 30 s, so the campaign is a large share of the traffic it overlaps: a median 34–45% of a window’s origins and 50% of the alarm’s cluster. A campaign that is a small share of the window appears only in the production injection at a tenth of the window.

Production traffic. We use eight days of four endpoints of Azion’s own services, real-user monitoring beacons (E1), a web console (E2), an API (E3) and single sign-on (E4), whose median windows hold about 3 000, 62, 41 and 20 origins. They are the four of the 12 exported that carry TLS, with at least 50 calibration windows and a median window of at least $k_{\min}$ origins. The exports count distinct origins and connections per fingerprint and five-minute window, and same-/24 pairs per window. The scope runs behind the operator’s WAF, so the profiles and floors describe the clients it lets through. We take the calibration days as attack-free: an attack among them would lower the level and raise the gate, making the scope more conservative. Into each window of the last five days, tested with the profile, thresholds and level of the days before them, we inject a botnet built as the generator builds one. It holds 25 to 1 000 attackers, or from a tenth of the median window to all of it. It puts 90% of its attackers on 25 stacks, uniformly, and the rest on one-off fingerprints that no scope names, so at most about 90% can be blocked. Its stacks are absent from the traffic (new), drawn from the profile past its ten most common fingerprints (shared), or the endpoint’s 25 most common fingerprints (adversarial). A flash crowd adds 100 or 1 000 users drawn from the day’s own clients to one window. A ninth day, the held-out day, was analyzed with the configuration fixed in advance.

B. Baselines, Configurations and Metrics

On generated traffic the scopes are compared on the alarm’s cluster: the modal fingerprint, the unseen filter, the z -score ($z > 3$) and the binomial test. The learned baseline is a

¹Measured 22 August 2026 from an access log the platform already collects. The extract holds only client-software fingerprints and their frequencies, with no address, host, URI, header or locale.

TABLE II
THE CONFIGURATIONS EVALUATED ON PRODUCTION TRAFFIC.

Configuration	Gate	Scope	Background	Exempt	Chosen	Held-out day
Base rule	$\Omega \geq \tau$	test	binomial	—	test days	in protocol
Binomial	origins	test $\cup$ unseen	binomial	fleets	test days	in protocol
z -score	origins	$z > \text{its p99}$	binomial	—	test days	outside ^a
Beta-binomial	origins	test $\cup$ unseen	beta-bin.	—	post hoc	post hoc*

*Post hoc: built after the held-out day was read. ^aIn the code before the day was read, but outside its protocol.

Random Forest trained on the campaign’s labels over the cross-session evidence of the three sub-relations (configuration (d) of Appendix C), forced onto zero false positives, the test’s operating point on every row but the adversarial one. On production traffic the configurations are those of Table II. Their gate is $\Omega \geq \tau_{\text{cluster}}$ or distinct origins at their 99th percentile, and their thresholds, levels and known fleets are fitted on attack-free days, in sample or cross-fitted. Cross-fitting refits them with every calibration day judged against a profile of the other days, as a test day is. Five choices were made on the test days: counting origins, calibrating λ_e , the gate, the union with the unseen filter and the 5% fleet share. Of four shares, only 5% had no more false alarms than the base rule on the first three test days, although 0.5% blocked about 25 times fewer legitimate clients and more of a 100-attacker botnet (Appendix E). $\rho = 3$, $k_{\min} = 5$, the 99th percentile of τ_{cluster} and the 25-stack botnet were fixed on generated traffic before production data were read.

Beyond classification metrics we report **collateral damage**, the share of legitimate traffic inside the scope: the false-positive rate of the filter an operator would deploy. On production traffic each configuration has a false-alarm rate (FA), the share of clean windows where it names a filter (5 643 on the test days, 1 152 on the held-out day), with an exact 95% interval, which assumes independent windows although misfires cluster by day. It also has a collateral per alarm (Coll.), the share of that window’s clients the filter blocks, which the tables give as a median over alarms. The tables also give the rate of the scope without the gate (No gate, a diagnostic) and with a flash crowd of 1 000 users in the window (Fl. 1k), and the share blocked of botnets of a median window or a tenth of it ($1 \times$, $0.1 \times$). The rate times the mean collateral is the expected share of legitimate clients blocked per clean window. The blocked share of the injected botnet is averaged over all windows, those where the trigger does not fire included. It is the product of how often the gate fires, how often the scope names the botnet when it does, and the share of attackers the named filter covers.

V. RESULTS

After the mechanism on generated traffic, production traffic gives three findings: calibration keeps false alarms low but sets a floor, the trigger decides what is stopped, and the evidence beyond the test days is thin.

TABLE III
THE ENRICHMENT SCOPE OF THE LARGEST- Ω CLUSTER AS A DETECTOR.

Scenario	Binomial test			Other scopes			Learned, FPR= 0	
	Recall	FPR	F_1	Modal	z -score	Unseen	(d)	+ profile
$M=1$	89.8%	0.00%	0.946	89.8%	89.8%	89.8%	91.7%	85.9%
$M=5$	90.0%	0.00%	0.948	0.0%	90.0%	90.0%	88.6%	91.7%
$M=25$	90.3%	0.00%	0.949	0.0%	90.3%	90.3%	36.4%	87.4%
$M=100$	38.6%	0.00%	0.556	0.0%	80.4%	88.1%	17.6%	86.9%
$M=25$, shared	85.4%	2.23%	0.910	0.0%	89.9%	0.0%	—	—
$M=25$, adversarial	30.4%	3.78%	0.452	3.6%	57.3%	0.0%	7.8%	6.6%

Recall of each scope alone, $K = 1000$, 15 seeds. Collateral of the other scopes: Section V-A. The learned models skip shared stacks.

A. The Scope on Generated Traffic

Here the sessions matching a scope *are* the flagged set, with no training or label (Table III, 15 campaigns), while per-session detection sits at chance, as the stealth mode imposes (Appendix C).

The natural scope fails. For a monolithic botnet the modal fingerprint and the test agree, blocking 89.8% of the attack with no collateral observed, where the fallback, a challenge to every client, would burden every legitimate user. From five stacks on, as Section III-C predicts, the modal fingerprint is a *legitimate* one: the frequency rule blocks 0.0% of the attack and 39.0% of legitimate traffic, 61.1% with a more concentrated benign population ($\alpha = 2.0$).

Profile-relative scopes succeed. The binomial test and the z -score block 90.0% of the attack at five stacks and 90.3% at 25, with **no collateral observed** across the campaigns, the rest sitting on one-off fingerprints: scoped mitigation trades completeness for precision. At 100 stacks a stack of about ten sessions among two thousand cannot be told from the tail of a 1 000-session profile, and the test falls to 38.6%, below the z -score’s 80.4% and the unseen filter’s 88.1%. A profile pooled over 30 attack-free periods restores it to 89.6%. The unseen filter, at a mean collateral of 0.03% (at most 0.5% in one seed), matches or beats the test because generated stacks never occur in the benign vocabulary, but blocks none of the shared stacks, which real clients also present. On those the test blocks 85.4% at 2.23% collateral and the z -score 89.9% at 3.37%.

The learned model needs labels that no operator holds during an attack, and at 25 stacks the test pulls ahead of it, 90.3% against 36.4% at zero false positives. Given the profile as two per-session features it recovers 87.4% and 86.9% at 25 and 100 stacks, trained on labels of the very campaign it scores. Allowed 1% false positives, which the test does not need, it reaches 66.8% at 25 stacks, and 94.6% with the profile.

Two conditions bound the result. When the botnet adopts common benign fingerprints, the boundary of the threat model, little is enriched: the test blocks 30.4% of the attack at 3.78% collateral, the z -score 57.3% at 11.3%, the frequency rule 3.6% at 39.0% and the learned model 23.4% at 1%. A flat or missing profile makes every fingerprint look rare, and collateral jumps to 77.6%, while moderate drift is tolerable. Per window, $\Omega \geq \tau_{\text{cluster}}$ alone also fires on flash crowds [32]. The

TABLE IV
THE CONFIGURATIONS ON PRODUCTION TRAFFIC, ALL ENDPOINTS.

Scope	Clean windows				New stacks		Shared	
	FA	No gate	Coll.	Fl. 1k	0.1×	1×	0.1×	1×
<i>Test days, calibrated in sample</i>								
binomial	0.1	2.2	32.9	22.2	3.0	19.9	1.9	17.6
beta-bin.*	0.4	4.4	10.5	28.4	3.3	31.4	2.7	24.7
z-score	0.4	6.1	10.8	13.1	3.1	28.8	2.3	18.9
<i>Test days, cross-fitted*</i>								
binomial	0.1	1.9	31.4	20.7	3.0	19.8	1.5	17.4
beta-bin.*	0.1	1.1	2.0	10.9	3.0	23.3	2.1	17.5
z-score	0.2	2.5	11.0	5.5	3.0	20.1	2.0	16.0
<i>Held-out day, calibrated on the eight days before it^a</i>								
rule	0.2	8.7	41.2	24.4	0.0	17.3	0.0	16.6
binomial	0.2	0.5	41.2	24.5	0.0	20.4	0.0	17.0
beta-bin.*	0.0	1.6	—	26.6	0.0	32.5	0.0	26.2
z-score	0.2	2.8	41.2	0.3	0.0	31.4	0.0	19.7

In %, FA, No gate, Fl. 1k: clean windows. Coll.: alarms. Blocked: all attack windows. *Post hoc. ^aNo gate, Fl. 1k, z-score: outside the protocol.

TABLE V
CALIBRATION FLOOR OF THE CONFIGURATIONS.

Endpoint	n_0	λ_e		Binomial		Beta-binomial*	
		bin.	beta*	new	shared	new	shared
E1, RUM beacons	2983	10^{-21}	10^{-4}	139/139	254/286	85/139	86/167
E2, web console	62	10^{-73}	10^{-2}	139/139	1090/1041	56/114	84/168
E3, API	41	10^{-11}	10^{-2}	139/139	168/227	56/114	56/140
E4, SSO	20	10^{-4}	10^{-2}	84/84	84/114	56/56	84/84

n_0 : median window. λ_e in sample. Floors in attackers per window, in sample / cross-fitted, test-day medians. *Post hoc, as is cross-fitting.

TABLE VI
THE CONFIGURATIONS ON PRODUCTION TRAFFIC, PER ENDPOINT.

Scope	Clean windows				New stacks		Shared	
	FA	No gate	Coll.	Fl. 1k	0.1×	1×	0.1×	1×
E1 binomial	0.0	1.0	—	0.3	11.8	70.8	7.7	68.6
E2 binomial	0.3	4.4	31.4	20.7	0.0	5.0	0.0	0.0
E3 binomial	0.1	1.8	48.5	33.8	0.0	1.9	0.0	0.5
E4 binomial	0.0	1.5	—	34.1	0.1	2.1	0.0	1.5

As Table IV, test days, calibrated in sample. Coll.: median of 4 alarms on E2, of at most 1 elsewhere.

test removes those alarms for generated crowds (Appendix E), though not for crowds of production clients (Section V-B).

B. Production Traffic: False Alarms and the Floor

False alarms. Tables IV to VI give the results. Pooled over them, every calibrated component exceeds its 1% target on the days after the calibration: the binomial configuration’s scope alone names a filter in 2.2% of clean test-day windows, and the distinct-origin gate alone fires in 3.0%, 5.4% on the busiest endpoint. The configuration needs both and fires on 0.1% of clean windows (5 of 5643, 95% interval 0.03–0.21%), 4 of them on the web console. A fleet raises the origin count and enriches its own fingerprint at once, but here the 5 joint misfires, against 3.5 under independence, are within chance ($P = 0.27$), unlike those of the post hoc beta-binomial and the z-score (Appendix E). The configuration’s misfires block a median 32.9% of the window’s clients, about 0.03% of legitimate clients per clean window. A crowd of 1000 users drawn from the day’s clients triggers it in 22.2% of windows,

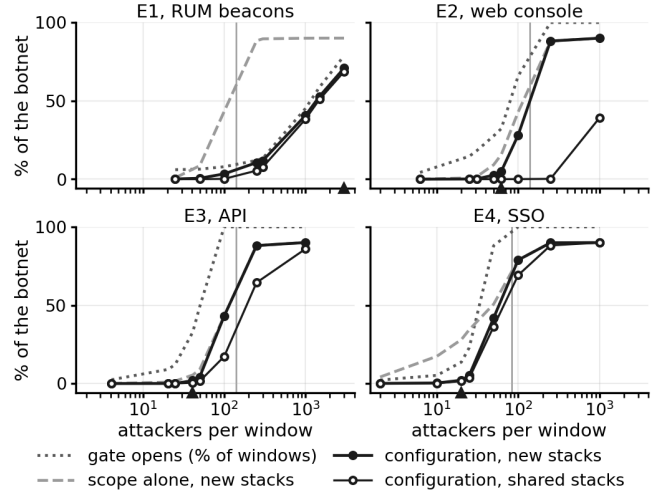


Fig. 2. What the binomial configuration stops, by the size of a 25-stack botnet, test days. Gate: windows where the distinct-origin gate opens. Scope alone: attackers the scope would block without the gate. Vertical line: floor on new stacks (Table V). Triangle: median window.

with lighter filters that block a median 4.3% of the window’s clients.

What the scope can name: the calibration floor. On the busiest endpoint fleets push λ_e to 10^{-60} , and exempting its five or six known fleets raises it to about 10^{-21} . Joined with the unseen filter, the binomial configuration names a 25-stack botnet on new stacks from 139 attackers per window on E1 to E3, a bound $k_{\min}$ sets, and from 84 on E4, where the test names them first (Table V). On stacks real clients also present, mostly rare ones past the profile’s ten most common fingerprints, the level binds, and the floor takes one at their median prevalence. The botnet is then named only past 254 attackers on the busiest endpoint, 8% of its window, and past 4 to 19 times the median window on the small ones, as medians of the daily ratios. The floor grows with the stacks, from 2% of the busiest window at 5 stacks to 34% at 100, and with the fingerprints’ rank, those of rank 11 to 35 being never named on the two largest endpoints (Appendix E). It is no cliff: below it the scope names the stacks chance makes larger, as a binomial model of stack sizes predicts (43.1% against 43.2% measured at 100 attackers on the busiest endpoint, Fig. 2).

Past the ratio limit the scope names nothing. The 4 to 28 most common fingerprints of each endpoint, which carry 86–94% of its origins, lie past that limit for a 25-stack botnet. A botnet on the endpoint’s 25 most common fingerprints, the boundary of the threat model, is thus mostly missed: the configuration stops 7.0% and 23.8% of 100 and 1000 attackers, and none on the two largest endpoints.

C. What Is Stopped: the Trigger Decides

A named botnet is stopped only in the windows where the distinct-origin gate also fires (Fig. 2). On the busiest endpoint the gate opens in 6–13% of the windows of botnets from 25 attackers up to a tenth of the window. The binomial

TABLE VII
THE SCOPES BEHIND EACH TRIGGER ON PRODUCTION TRAFFIC.

Trigger	Scope	FA, E1		FA, E2-E4		New		Shared		Fl. 1k	Coll.
		test	held	test	held	test	held	test	held		
Origin gate	binomial	0.00	0.00	0.28	0.69	11.8	0.0	7.7	22.2	32.9	
	beta-bin.*†	0.14	0.00	0.28	0.00	11.9	0.0	8.3	10.9	2.0	
	unseen	0.00	0.00	0.07	0.00	11.8	0.0	0.0	3.4	3.3	
Seasonal gate*	binomial	0.00	0.00	2.64	0.69	20.4	15.3	14.4	22.3	35.2	
	beta-bin.†	0.07	0.00	0.21	0.00	20.4	15.3	15.1	10.9	16.9	
	unseen	0.00	0.00	0.00	0.00	20.4	15.3	0.0	3.4	–	
No gate*	binomial	0.97	0.00	4.38	1.04	89.6	89.7	61.1	22.4	32.6	
	beta-bin.†	0.90	1.74	1.46	1.39	89.7	90.1	65.7	11.0	6.6	
	unseen	0.28	0.00	0.42	0.35	89.6	89.7	0.0	3.4	5.7	

In %. Held: held-out day. FA per endpoint. E2–E4 their highest. Fl. 1k, Coll.: pooled. New, Shared: E1’s tenth-size botnet. †Cross-fitted. *Post hoc.

configuration therefore stops 3.4% of a botnet of 100 attackers and 11.8% of a botnet a tenth the size of the window, 0.6–37.2% depending on the day, of which its scope alone would block 43.2% and 89.6%. On the small endpoints a botnet of 100 attackers, 1.6 to 5 times their median window, opens the gate in 66–100% of the windows, and there the floor decides.

Other triggers. Two triggers, examined after the held-out day was read, locate the loss (Table VII and Fig. 3, with the binomial in sample, as the protocol fixed it). A seasonal gate compares the distinct origins with their median at the same hour of the calibration days, at the same 1% budget. It stops more of a small botnet on the busiest endpoint, but of the binomial and the beta-binomial, each in sample and cross-fitted, only the cross-fitted beta-binomial keeps the console within the budget behind it. The scope as its own trigger stops about 90% of that botnet on every day. Under the binomial with known fleets its rate on the busiest endpoint, 0.97% (95% interval 0.53–1.63%), sits at the budget and rose from no misfire on the first two test days to 8 of 288 on the last. Its misfires there block a median 1.0% of the clients, and without the fleet exemption it fires in 92 of the 1 440 test-day windows and 95 of the held-out day’s 288. On new stacks of at least $k_{\min}$ origins, as in the tenth-size botnet, the unseen filter by itself does as well under every trigger, since it names each such stack, with fewer false alarms and flash-crowd firings. As its own trigger its misfires also cluster by day (Appendix E). The test adds the shared stacks, and smaller new ones on the small endpoints only. Of 100 attackers on new stacks, the cross-fitted beta-binomial behind the seasonal gate stops 53–84% there and 3.9% on the busiest endpoint, against 43% for the unseen filter as its own trigger.

D. What the Evidence Supports

The held-out day. Exported after the configuration was fixed and analyzed with metrics fixed in advance (Appendix F), the held-out day gives the binomial configuration 2 false alarms in 1 152 clean windows ($P = 0.27$, and up to 3 would have passed). The test is weak. Beyond the protocol’s metrics, a tripled rate would be flagged 37% of the time, and the day passed mostly because the gate was quiet: it opened in only 3 clean windows, 0.26% against 3.0% on the test days. The scope misfired in two of them, both on the web

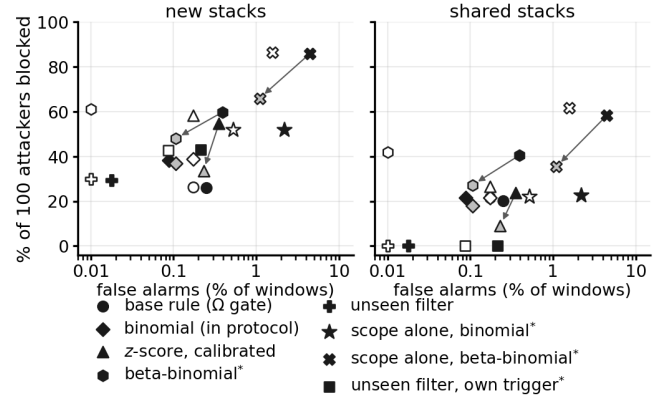


Fig. 3. Operating points on production traffic, pooled over the endpoints, where the small ones dominate, 100 attackers per window, zero rates at 0.01%. Filled: test days. Open: held-out day. Grey: cross-fitted*, arrowed from in sample. *Post hoc.

console, more often than the test days’ rate given the gate predicts ($P = 0.003$). Flash crowds of 100 users trigger the configuration in 1.1% of windows, and at a tenth of the busiest window the gate never opened. On the single sign-on, crowds of 1 000 users, also beyond the protocol, trigger it in 97.2% of windows, against 34.1% on the test days, nearly all on one fingerprint far more common that day (Appendix E).

Post hoc analyses. The beta-binomial and a cross-fitted calibration were built after the held-out day was read, so their results are post hoc. The beta-binomial’s floor on shared stacks falls to 86 attackers on the busiest endpoint (3%) and to 1.4 to 4.2 windows elsewhere. In sample 19 of its 22 false alarms fall on the web console, where 74.4% of 1 000-user crowds trigger it. On the held-out day it raised no false alarm. A first design of the cross-fitting, which calibrated on the last calibration day alone, was dropped after its results were seen: one day misses the rare fleet events that set the level. The binomial configuration barely moves, with 6 false alarms against 5. The calibrated z -score’s lead in detection came from its in-sample threshold, which cross-fitting raises by a median factor of 1.8. Only the beta-binomial nearly meets its target out of sample: its scope alone fires in 1.1% of clean windows, and the console’s 1 000-user crowds trigger it in 16.1% of windows. Its floor on the rare shared stacks of Table V is 6% of the busiest window and at most 4.4 windows elsewhere, below the binomial’s cross-fitted floor on every endpoint, though not everywhere on more common stacks against the in-sample binomial (Appendix E).

This WAF’s verdicts cannot serve as labels. The operator’s WAF blocked a median 9%, 63% and 39% of each window’s clients on E2 to E4, and none on E1. With the profile built without those clients, the scopes’ agreement with the WAF is built in: a stable fingerprint is enriched there only if 70–88% of its clients were blocked. With a profile of all clients, 0.4–33% of the clients the three profile-relative scopes would block were blocked, less than a random pick (10–63%) on every endpoint. On the API and single sign-on

they catch at most 2 of the WAF’s 38 surges (Appendix E). The verdicts cannot validate the scope, which reads deviations from everyday traffic and does not recover what this WAF blocks.

VI. DISCUSSION AND LIMITATIONS

Which configuration to recommend. The floor and the trigger decide per endpoint. On the three small endpoints the binomial configuration names shared stacks only past 4 to 19 windows and new stacks from 2 to 4 windows, so below those sizes the fallback is a rate limit or a challenge to every client. On the busiest endpoint the scope names a botnet on new or rare shared stacks under a tenth of its window, but the distinct-origin gate lets most of it through (Table VII). Only the binomial configuration behind that gate was tested on a day it had not seen, as fixed in advance, so it is the configuration we would deploy there now, although it stops few small botnets there, none on the held-out day. The next test, pre-specified on new days over a weekly cycle with an external timestamp, would take the unseen filter as its own trigger as the baseline. It matched every scope on new stacks of at least $k_{\min}$ origins within the budget everywhere, pooled over the days (Table VII), and the test would cover 100 attackers as well as a tenth of the window. Against it would run the cross-fitted beta-binomial behind the seasonal gate, which gains on shared stacks everywhere and on smaller new stacks on the small endpoints, but loses most new-stack detection on the busiest one (Table VII). On the busiest endpoint the binomial scope with known fleets as its own trigger would run too, at the budget there but rising by day and resting on the fleet exemption. Whether a campaign at the floor exhausts the service depends on its headroom, which we did not measure.

Each evaluation establishes one thing. The generator shows the mechanism under a mimicry imposed by construction. Its campaigns, a third to half of the window, are easier than the tenth at which production shows the gate failing, and its stacks never occur in the benign vocabulary, which is why an unseen filter matches the scope there. Production traffic measures false alarms on real clients and detection only for an injected botnet of one shape, 25 uniform stacks, new, mostly rare or the endpoint’s most common, over nine days of one operator’s own endpoints, too few to calibrate a weekly cycle. The five choices of Section IV-B were made on the days they are reported on, so only the held-out day holds them out, and the cross-fitted calibration removes only the levels’ in-sample fit. None of the evaluations covers a captured stealthy campaign, and Appendix C gives further limits.

The scope depends on an observable discriminator. A bot outside the threat model that presents a common fingerprint defeats it (Section V-B). Encrypted Client Hello [33] hides JA4 from on-path observers but not from a defender that terminates TLS, as assumed here. The profile must be refreshed: a release that gives a large population a new fingerprint would reach the unseen filter at once, which the nine days did not show. Neither a flash crowd on one client type, the harder case, nor an attacker shaping the profile or the fleet exemption was

injected, so profiles should be rebuilt from windows the scope did not name.

What exchange loses, and what the ontology adds. Of the standards we examined, STIX 2.1, DOTS and Flowspec have no JA4 property in their core vocabulary, and OCSF records JA4 fingerprints in network events and in the evidence of findings [11] but defines no filter or remediation over them. The scope’s STIX 2.1 bundles pass the OASIS validator and cross a TAXII 2.1 server (Appendix F). MISP’s importer drops the fingerprints, which STIX 2.1 carries only in an extension the importer does not map to MISP’s own JA4 object. The ACL model that DOTS filters mimic matches a TLS client’s parameters since a 2025 extension, but names no fingerprint, and DOTS defines its own filters without extending it [34]. Fingerprint-scoped mitigation thus needs a JA4 match in the DOTS filters and a JA4 property in STIX, for instance in a TLS extension of its network-traffic object. The ontology adds nothing to the detection numbers or to the trigger. It is the specification the operator deploys, carried into the evidence chain and compiled into the query a log store runs, which reproduced the exported origin and /24-pair counts, and the JA4 pair counts where the WAF blocked no client (Appendix F). Kept as classes, the equality sub-relations make admission constant and aggregation linear (Appendix D).

VII. CONCLUSION AND FUTURE WORK

We treated the scope of an application-layer DDoS mitigation as a calibrated statistical decision over the TLS fingerprints of an endpoint’s clients, specified in a session ontology. On generated stealthy campaigns the alarm’s most common fingerprint is a legitimate one once the botnet spans several TLS stacks, while profile-relative scopes block 90% of botnets of up to 25 stacks absent from legitimate traffic, with no observed collateral and no label. On a CDN operator’s traffic, legitimate fleets set a calibration floor: the binomial configuration names a 25-stack botnet on rare fingerprints real clients share only past 8% of the busiest endpoint’s window and 4 to 19 windows on the small ones. Its false alarms stay at 0.1% of clean windows on the days its configuration was chosen on, consistent with a held-out day on which its gate barely opened, though the scope misfired in 2 of the 3 windows where it did. On the busiest endpoint its gate lets most of a small botnet through. Post hoc, the unseen filter as its own trigger stops 90% of such a botnet on new stacks of $k_{\min}$ origins or more within the budget. A cross-fitted beta-binomial behind a seasonal gate, also post hoc, stops 15% of it on shared stacks but gives up most new-stack detection. No standard we examined expresses such a scope in its core vocabulary. Next come a pre-specified test of these triggers on new days against the unseen filter, HTTP/2 attacks, and a captured stealthy campaign.

ACKNOWLEDGMENT

We thank Azion Technologies (<https://www.azion.com/>) for authorizing the measurements and traffic statistics used here.

REFERENCES

- [1] J. Snellman and D. Iamartino, “How it works: The novel HTTP/2 ‘Rapid Reset’ DDoS attack,” Google Cloud Blog, Oct. 2023, accessed Sep. 25, 2026. [Online]. Available: <https://cloud.google.com/blog/products/identity-security/how-it-works-the-novel-http2-rapid-reset-ddos-attack>
- [2] E. Cambiaso, G. Papaleo, G. Chiola, and M. Aiello, “Slow DoS attacks: Definition and categorisation,” *International Journal of Trust Management in Computing and Communications*, vol. 1, no. 3–4, pp. 300–319, 2013.
- [3] C. Kemp, C. Calvert, T. M. Khoshgoftaar, and J. L. Leevy, “An approach to application-layer DoS detection,” *Journal of Big Data*, vol. 10, no. 1, p. 22, 2023.
- [4] M. Odusami, S. Misra, O. Abayomi-Alli, A. Abayomi-Alli, and L. Fernández-Sanz, “A survey and meta-analysis of application-layer distributed denial-of-service attack,” *International Journal of Communication Systems*, vol. 33, no. 18, p. e4603, 2020.
- [5] L. Belcastro, C. Carlucci, C. Cosentino, P. Liò, and F. Marozzo, “Enhancing network security using knowledge graphs and large language models for explainable threat detection,” *Future Generation Computer Systems*, vol. 176, p. 108160, 2026.
- [6] R. Mahajan, S. M. Bellovin, S. Floyd, J. Ioannidis, V. Paxson, and S. Shenker, “Controlling high bandwidth aggregates in the network,” *ACM SIGCOMM Computer Communication Review*, vol. 32, no. 3, pp. 62–73, 2002.
- [7] A. G. Alcoz, M. Strohmeier, V. Lenders, and L. Vanbever, “Aggregate-based congestion control for pulse-wave DDoS defense,” in *Proc. ACM SIGCOMM*, 2022, pp. 693–706.
- [8] F. Soldo, K. Argyraki, and A. Markopoulou, “Optimal source-based filtering of malicious traffic,” *IEEE/ACM Transactions on Networking*, vol. 20, no. 2, pp. 381–395, 2012.
- [9] M. Boucadair and T. Reddy.K, “Distributed Denial-of-Service Open Threat Signaling (DOTS) Data Channel Specification,” Internet Engineering Task Force, RFC 8783, May 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8783>
- [10] C. Loibl, S. Hares, R. Raszkun, D. McPherson, and M. Bacher, “Dissemination of Flow Specification Rules,” Internet Engineering Task Force, RFC 8955, Dec. 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8955>
- [11] OCSF Project, “Open cybersecurity schema framework (OCSF), version 1.3.0,” GitHub repository, Aug. 2024, adds the `ja4_fingerprint` object to network events. Accessed Sep. 27, 2026. [Online]. Available: <https://github.com/ocsf/ocsf-schema>
- [12] C. Dietzel, M. Wichtlhuber, G. Smaragdakis, and A. Feldmann, “Stellar: Network attack mitigation using advanced blackholing,” in *Proc. ACM International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, 2018, pp. 152–164.
- [13] F. Silveira and C. Diot, “URCA: Pulling out anomalies by their root causes,” in *Proc. IEEE INFOCOM*, 2010.
- [14] D. Brauckhoff, X. Dimitropoulos, A. Wagner, and K. Salamati, “Anomaly extraction in backbone networks using association rules,” in *Proc. ACM Internet Measurement Conference (IMC)*, 2009, pp. 28–34.
- [15] Z. Li, M. Sanghi, Y. Chen, M.-Y. Kao, and B. Chavez, “Hamsa: Fast signature generation for zero-day polymorphic worms with provable attack resilience,” in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2006.
- [16] S. Kandula, D. Katabi, M. Jacob, and A. Berger, “Botz-4-Sale: Surviving organized DDoS attacks that mimic flash crowds,” in *Proc. USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2005.
- [17] R. Bharathi and R. Sukanesh, “A PCA based framework for detection of application layer DDoS attacks,” *WSEAS Transactions on Information Science and Applications*, vol. 9, no. 12, pp. 389–398, 2012.
- [18] A. Martinetti, “Advancing threat intelligence: JA4 fingerprints and inter-request signals,” Cloudflare Blog, Aug. 2024, accessed Sep. 25, 2026. [Online]. Available: <https://blog.cloudflare.com/ja4-signals/>
- [19] A. Lakhina, M. Crovella, and C. Diot, “Mining anomalies using traffic feature distributions,” in *Proc. ACM SIGCOMM*, 2005, pp. 217–228.
- [20] S. Ranjan, R. Swaminathan, M. Uysal, A. Nucci, and E. Knightly, “DDoS-Shield: DDoS-resilient scheduling to counter application layer attacks,” *IEEE/ACM Transactions on Networking*, vol. 17, no. 1, pp. 26–39, 2009.
- [21] G. Fernandes Jr., J. J. P. C. Rodrigues, and M. L. Proença Jr., “Autonomous profile-based anomaly detection system using principal component analysis and flow analysis,” *Applied Soft Computing*, vol. 34, pp. 513–525, 2015.
- [22] Y. Xie and S.-Z. Yu, “Monitoring the application-layer DDoS attacks for popular websites,” *IEEE/ACM Transactions on Networking*, vol. 17, no. 1, pp. 15–25, 2009.
- [23] P. E. Kaloroumakis and M. J. Smith, “Toward a knowledge graph of cybersecurity countermeasures,” The MITRE Corporation, 2021, accessed Sep. 25, 2026. [Online]. Available: <https://d3fend.mitre.org/resources/D3FEND.pdf>
- [24] K. Liu, F. Wang, Z. Ding, S. Liang, Z. Yu, and Y. Zhou, “Recent progress of using knowledge graph for cybersecurity,” *Electronics*, vol. 11, no. 15, p. 2287, 2022.
- [25] M. Husák, M. Čermák, T. Jirsík, and P. Čeleda, “HTTPS traffic analysis and client identification using passive SSL/TLS fingerprinting,” *EURASIP Journal on Information Security*, vol. 2016, p. 6, 2016.
- [26] B. Anderson and D. McGrew, “TLS beyond the browser: Combining end host and network data to understand application behavior,” in *Proc. ACM Internet Measurement Conference (IMC)*, 2019, pp. 379–392.
- [27] X. Li, B. Amin Azad, A. Rahmati, and N. Nikiforakis, “Good bot, bad bot: Characterizing automated browsing activity,” in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2021, pp. 1589–1605.
- [28] J. Althouse, “JA4+ network fingerprinting,” FoxIO Blog, Sep. 2023, accessed Sep. 25, 2026. [Online]. Available: <https://blog.foxio.io/ja4+-network-fingerprinting>
- [29] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, “Understanding the Mirai botnet,” in *Proc. 26th USENIX Security Symp.*, 2017, pp. 1093–1110.
- [30] S. Frolov and E. Wustrow, “The use of TLS in censorship circumvention,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2019.
- [31] OASIS, “STIX version 2.1,” OASIS Standard, 2021, accessed Sep. 25, 2026. [Online]. Available: <https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html>
- [32] J. Jung, B. Krishnamurthy, and M. Rabinovich, “Flash crowds and denial of service attacks: Characterization and implications for CDNs and web sites,” in *Proc. 11th Int. World Wide Web Conf. (WWW)*, 2002, pp. 293–304.
- [33] E. Rescorla, K. Oku, N. Sullivan, and C. A. Wood, “TLS Encrypted Client Hello,” Internet Engineering Task Force, RFC 9849, Mar. 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9849>
- [34] T. Reddy.K, D. Wing, and B. Anderson, “Manufacturer Usage Description (MUD) for TLS and DTLS Profiles for Internet of Things (IoT) Devices,” Internet Engineering Task Force, RFC 9761, Apr. 2025. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9761>
- [35] M. Shafi, A. H. Lashkari, V. Rodriguez, and R. Nevo, “Toward generating a new cloud-based distributed denial of service (DDoS) dataset and cloud intrusion traffic characterization,” *Information*, vol. 15, no. 4, p. 195, 2024.
- [36] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [37] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, “CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment,” *Sensors*, vol. 23, no. 13, p. 5941, 2023.

APPENDIX A

SUB-RELATION SPECIFICATION

The rule’s coordination mass is

$$\Omega(S) = \sum_{i \in \mathcal{R}_{\text{rel}}} w_i |E_i(S)|, \quad (1)$$

where $E_i(S)$ is the set of pairs of distinct origins $\{o(s_a), o(s_b)\}$ whose sessions sub-relation i links, $o(s)$ being the source address of session s . Each pair counts once, so one client’s many connections cannot pose as coordination. The weight w_i follows the attacker’s cost of breaking the signal,

from the TLS fingerprint (1.0) through endpoint convergence (0.6) to network proximity (0.3), since carrier-grade NAT makes a shared /24 a poor discriminator, and calibration supports only the top of that order (Appendix B). For an equality sub-relation $|E_i(S)| = \sum_k \binom{n_k}{2}$ over the class sizes n_k in origins, so Ω is a function of the class sizes the compiled query returns (Section III-E). For each sub-relation, the procedure below decides whether a candidate pair (s_a, s_b) is linked.

TLSFingerprint: exact equality of the full JA4, the relation exercised here. A near variant, annotated *ja4_distance = 1* (one symbol of difference in the ordered cipher or extension blocks), would absorb version drift within one client library, but it is not an equivalence relation and would need pair edges, so it is specified but not exercised. **ReusedIdentity**, **TemporalPattern** and **PayloadSignature**, specified but not exercised, match a shared cookie, token or username, inter-arrival sequences within a DTW distance τ_{DTW} , and payload summaries above a cosine similarity $\tau_{payload}$. **EndpointConvergence:** the same endpoint under path-pattern normalization (`/api/users/12345` $\rightarrow$ `/api/users/{id}`), annotated *exact = true* on an exact-path match. **NetworkProximity:** a shared IPv4 /24 (IPv6 /48), a shared ASN from current BGP tables, or both (*strong = true*).

APPENDIX B WEIGHT CALIBRATION

On the scenarios of our earlier runs, a near-uniform benign vocabulary against a monolithic botnet, maximizing the per-session ROC AUC of $w_{tls} z(ja4) + w_{ep} z(endpoint\ conv.) + w_{net} z(net\ prox.)$ over the grid $\{0.3 \dots 1.0\}^3$ (125 combinations, 60 scenarios) gives (1.0, 0.3, 0.3). Only the TLS fingerprint discriminates on its own (AUC 0.93, against 0.50 for endpoint convergence and 0.58 for network proximity), and every endpoint weight of the grid attains the same optimal AUC of 0.943, as do the ontology’s weights (1.0, 0.6, 0.3). Only the top of the order is thus identified, and the per-window rule detects as well with uniform weights (Appendix E). The canonical scenario’s pool of 2000 benign fingerprints takes the Zipf shape of the 495 measured at one point of presence and exceeds the 76–818 each production endpoint shows. The popularity measurement is weighted per request, which if anything overstates concentration.

APPENDIX C ADDITIONAL LIMITATIONS

Instrumentation and deployment. Partial or noisy extraction of sessions, identities and fingerprints degrades the approach. Counting origins by source address undercounts users behind a carrier-grade NAT and overcounts clients that rotate IPv6 privacy addresses, whose natural unit is the /64. The production run is offline, over aggregated counts per endpoint, so a live CDN deployment still needs the partitioning of Appendix D, which we did not implement.

Coverage. HTTP/2 attacks and other application protocols (DNS, gRPC) need sub-relations beyond the Slow HTTP DoS

family over HTTP/1.1, and $\Omega(S)$ instantiates only the three sub-relations the generator and captures expose, so the weights of the others are untested.

Dataset breadth. We found no independent capture of the stealthy regime: BCCC-cPacket-Cloud-DDoS-2024 criticizes the application-layer coverage of earlier datasets and ships 17 TCP-based DDoS scenarios [35].

Laboratory captures and KLAGe. CICIDS2017 [36] (Slowloris, Slowhttptest, HULK, GoldenEye) and CIC-IoT2023 [37] (DDoS Slowloris, HTTP Flood), on which KLAGe reports $F_1 = 84.1\%$ for DDoS Slowloris, run through the same extraction pipeline (PCAP $\rightarrow$ JA4 + flows $\rightarrow$ sessions). On CIC-IoT2023 DDoS Slowloris, whose attack clusters hold seven origins in one of 143 /24 prefixes, a strong per-session baseline (eight flow attributes) reaches 0.900 (AUC 0.987) and the cross-session representation 0.911 (AUC 0.982). Real Slowloris, unlike our stealthy campaigns, leaves a per-session flow signature. The comparison with KLAGe is *uncontrolled*: our strong baseline also beats the published 0.841, and the released code, which starts from a graph of unpublished construction and ships no weights, admits no controlled rerun.

Per-session detection and learned models. Each configuration of this ablation runs on 30 seeds, each campaign split 70/30 into training and test sessions. Configuration (a) uses per-session flow attributes alone, on a deliberately strong Random Forest (RF) repeated with gradient boosting, a perceptron and logistic regression. Configuration (b) adds per-session ontological attributes, (c) uses network proximity only, and (d) the cross-session evidence of the three sub-relations, the share of the cluster sharing one JA4 or /24 plus its size. We also reimplement PCA profiling [21], PCA with k -means over a behavioral matrix [17] and supervised RF/SVM over session features [3]. At $K = 50$ and $K = 1000$, (a) reaches a per-session ROC AUC of 0.498 and 0.503, (b) 0.500 and 0.502, the three academic baselines 0.495–0.518, and (a) stays at 0.488–0.503 under every classifier family ($\alpha = 1.5$, $M = 25$, 30 seeds), the chance level the stealth mode imposes. The cross-session representation (d) reaches 0.927 and 0.982, and (c), restricted to network proximity, 0.499 and 0.659 (paired Wilcoxon tests, Bonferroni-corrected, $p_{Bonf} = 7.5 \times 10^{-9}$ for (d)–(c) and (d)–(a) at $K = 1000$).

Learned representation. That separability does not transfer. Fragmenting the botnet over M stacks makes the cross-session evidence non-monotonic in the label: at $K = 1000$ an attacker shares a fingerprint with a few dozen peers, a legitimate client in the benign head with hundreds. Trained on five stacks and tested on 25 or 100, (d) falls to 0.61 and 0.63, and trained on 25 to 0.48 on five and 0.75 on 100, against 0.95–0.995 on the same M with other seeds. Cross- M tests use disjoint seeds, since the generator draws every benign session before anything M -dependent: a same-seed split would score (a) at 0.88–0.94 by memorization.

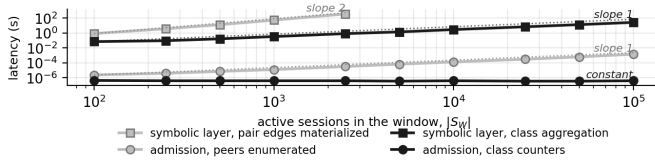


Fig. 4. Cost of both layers, with pair edges and with class counts.

```
"@type": "kg:CoordinatedHTTPFlood",
"kg:clusterSize": 1991, "kg:coordinationScore": 1303422.8,
"kg:activatedSubRelations": [
  {"@type": "kg:relatedByTLSPFingerprint",
   "kg:coordinationWeight": 1.0, "kg:linkedPairs": 114710},
  {"@type": "kg:relatedByEndpointConvergence",
   "kg:coordinationWeight": 0.6, "kg:linkedPairs": 1981045},
  {"@type": "kg:relatedByNetworkProximity",
   "kg:coordinationWeight": 0.3, "kg:linkedPairs": 286}],
"kg:derivedMitigationScope": {
  "kg:endpoint": "10.0.0.1:443",
  "kg:tlsJa4": ["t13d1516h2_synth_0001_00", ...]}
```

Listing 1. Evidence chain of a canonical campaign ($M = 25$, $K = 1000$), counted in origins. The scope lists 25 fingerprints, abridged.

APPENDIX D COST MODEL

Admission happens per request and the Ω aggregation once per window, so we time the two separately on a synthetic window: 30% of sessions in one campaign sharing a botnet JA4, benign JA4 from a pool of 2000, eight endpoints, dispersed /24s. Each runs with pair edges, as the rules are written, and with class counts for the equality sub-relations (medians of three repeats on one core), and both give the same $\Omega(S)$ to within 7×10^{-12} . The symbolic layer runs on *rdflib* in place of Jena/TDB2, so its figures are an upper bound.

With pair enumeration, median admission latency grows from $2.3 \mu\text{s}$ at $|S_W| = 100$ to $127 \mu\text{s}$ at 10 000. With class counters it stays between 0.33 and $0.42 \mu\text{s}$ from 100 to 100 000 sessions (Fig. 4, dotted lines are reference slopes). Materializing pair edges costs 0.84 s at $|S_W| = 100$ and 52.8 s at 1000. Class aggregation, one membership triple per session and sub-relation and a SPARQL query summing the weighted class counts, counting sessions (on the generator, one per origin), takes 0.33 s at 1000 sessions, 161 times faster, and 26.4 s at 100 000. The quadratic term thus lives in the value of $\Omega(S)$, which counts pairs, and in the pair edges of the non-transitive sub-relations.

Partitioning at CDN scale. The rule reads one endpoint per S , so partitioning by endpoint evaluates it *exactly*, with no cross-shard traffic. A campaign spreading one JA4 across many endpoints then accumulates too few pairs in any shard, which a global sketch, left unimplemented, would have to catch.

APPENDIX E THE RULE PER WINDOW AND PRODUCTION DETAILS

Here the rule runs as deployed, per window of $W = 300$ s and endpoint, and no threshold sees an attack or a label. τ_{cluster} is the 99th percentile of Ω over the windows of 30 attack-free

runs, which also form the profile. False alarms are counted on 30 more, detection on the canonical campaigns ($\alpha = 1.5$, $M = 25$) over windows with at least $k_{\min} = 5$ attackers, and a flash crowd retimes 25, 50 or 100 legitimate sessions of a third run into one window. We spread benign starts over 60 min, since the generator packs them into the first minutes.

Over 30 runs each, Ω passes τ_{cluster} in 77.8% and 85.7% of the 90 and 35 attack windows at $K = 1000$ and 50, in 0.6% of 360 clean windows and in 80–100% of the flash crowds of 25 to 100 users. A flash crowd adds mass as a campaign does. The test tells them apart because these crowds draw users from the benign mix the profile describes, while a surge on one client type is enriched, as production fleets show. With the test the rule fires on no clean window (one-sided 95% upper bound 0.8%) and no flash crowd, and blocks a median 75–80% of the attackers in every attack window Ω flags, hitting no legitimate session. With uniform weights Ω catches at least as many attack windows (78.9% and 85.7%), and without its endpoint term 33.3% and none, while still flagging 53–100% of flash crowds. A distinct-origin gate catches 82.2% and 85.7%, and with the test no clean window or flash crowd. With a profile of one run, 1 000 sessions, instead of thirty, the rule fires in 68.9% and 80.0% of attack windows and on one crowd of 25 users, at 11.7% collateral.

Production traffic. The exports carry no prefixes, so the scope is judged on JA4 alone. At the nominal level the scope names a filter in 60.1% of clean windows counted in connections, since one client opens many, and in 26.9% counted in origins, because legitimate fleets switch on together. Calibrating λ_e on the preceding days brings the base rule to 0.2%, and the distinct-origin gate, which fires with Ω in 136 of the 181 clean windows where either fires, halves its false alarms with no loss of detection. At $\lambda_e = 10^{-60}$ the binomial test names a stack only past 15 to 18 origins. Fleets, such as a probe fleet active only in business hours, push λ_e down, and without fleets exempted the test alone names a filter in 1.5–6.3% of the next day’s windows, often a fleet blocking about half of a window’s clients.

Details of Section V. In attack windows the calibrated scopes block a median 0% of legitimate clients and up to 71% in the worst window of a small endpoint. On the held-out day the base rule, the protocol’s reference, fired on the same two windows as the configuration, as did the calibrated z -score, outside the protocol. Pooled, the binomial configuration stops 38.4% and 77.7% of 100 and 1000 attackers on new stacks and 21.7% and 63.3% on shared ones, figures the small endpoints dominate. On the held-out day it stops 38.9% and 73.4% of 100 and 1000 attackers on new stacks and 21.5% and 60.9% on shared ones. Fingerprints the profile has never seen appear every day but rarely concentrate: 0.21% of the test days’ windows hold one with at least $k_{\min}$ origins. As its own trigger the unseen filter’s misfires cluster by day: all 4 on the busiest endpoint and both on the console fall on one day, and 5 of the API’s 6 on another. On the held-out day Table VII’s shared-stack cells read 0.0 (origin gate), 11.9, 14.3 and 0.0 (seasonal gate) and 66.1, 83.3 and 0.0 (no gate), and its flash-

crowd cells 24.5–25.1% for the calibrated scopes against 0.1% for the unseen filter. That day the single sign-on’s 1 000-user crowds also trigger the post hoc seasonal gate’s cross-fitted beta-binomial in 97.6% of windows, and the configuration’s filters there block a median 1.1% of the window’s clients. Gate and scope misfire together more often than independence predicts for the beta-binomial and the calibrated z -score, in 22 and 20 windows against 7.5 and 11.9 ($P < 10^{-4}$ and $P = 0.02$ for a Poisson count), and in 6 and 13 against 1.8 and 4.6 cross-fitted. For the binomial configuration the excess is within chance, 5 against 3.5 and 6 against 3.2 cross-fitted ($P = 0.27$ and 0.10). The expectation pools each endpoint’s days, so day-to-day swings alone can inflate the excess. The misfires of the other two also cluster by day, 13 of each on one day of the console, so exact intervals, which assume independent windows, understate their uncertainty. On the busiest endpoint, depending on the day, the gate opens in 0.7–42% of the windows of a botnet a tenth the size of the window. It opens in 45% at a third and in 79% at a whole window. Of 100 attackers on new stacks, the unseen filter by itself behind the origin gate stops 29.4%, the calibrated z -score 54.7% (23.7% on shared ones) and the post hoc beta-binomial 59.8% (40.5%). Under the binomial in sample, the shared-stack floor for 5 and 100 stacks is 50 and 1 026 attackers on E1, 164 and 6 158 on E2, 28 and 1 017 on E3 and 17 and 454 on E4. For 25 stacks, fingerprints of rank 11 to 35 need 808 on E3 and 114 on E4, and those of rank 36 to 100 need 743, 1 898, 227 and 84. On ranks 36 to 100 the busiest endpoint’s floor falls from 743 attackers under the binomial to 480 under the beta-binomial in sample, and rises to 949 cross-fitted. Against the binomial in sample, the cross-fitted beta-binomial’s floor on those ranks is also higher on the API (255 against 227), equal on the single sign-on and lower on the console (509 against 1 898). Cross-fitted, post hoc, the calibrated z -score stops 33.6% of 100 attackers against 54.7%. The beta-binomial’s levels then fall to between 10^{-17} and 10^{-2} , its scope alone fires in 1.2% of the held-out day’s windows, and one known fleet appears on the busiest endpoint on three of the five days. Most clients the WAF blocked sat on fingerprints that unblocked clients also present. With the profile built without them, 84–99% of the clients the scopes would block on the full traffic of the API and single sign-on were blocked by the WAF. The scopes catch the WAF’s surges clearly above chance only on the console, and there with under 1% of the blocked clients.

Other baselines. Uncalibrated and behind the Ω gate, the z -score fires on 2.6% of clean windows and 64.5% of flash crowds of 100 users (86.8% of 1 000). Flash crowds of 1 000 users trigger the base rule in 21.8% of windows. When they trigger the binomial configuration, its filters block a median 7.1%, 18.6% and 2.7% of the window’s clients on E2 to E4, and 20.6% at the 90th percentile.

Known fleets and the scope as a trigger. Of the four known-fleet shares tried on the first three test days, 5% was the only one whose false alarms did not exceed the base rule’s (5 against 6, the others 12 to 16). By the rate times the median collateral, a rougher measure than the mean, the

0.5% share blocked about 25 times fewer legitimate clients per clean window on those days (0.0028% against 0.070%). It also stopped more of 100 attackers on new stacks (44.6% against 30.9%). Post hoc, a trigger on the scope alone fires, with known fleets, on 2.2% of pooled clean windows and 0.5% on the held-out day, against 3.5% and 8.8% for the union without them. With known fleets it stops 61.1% of the busiest endpoint’s tenth-size botnet on shared stacks. Against the post hoc beta-binomial, under which no fingerprint qualifies as a known fleet in sample, the busiest endpoint’s level rises to about 10^{-4} and the others’ to the 0.01 cap. There the scope alone stops 90.0% and 78.3% of a botnet a tenth the size of the busiest window, on new and shared stacks, at 2.1% of that endpoint’s clean windows. The post hoc seasonal gate alone fires on 3.0% of clean test-day windows, as the origin gate does, and on the busiest endpoint stops 10.0–44.7% of a tenth-size botnet depending on the day. Setting $\rho = 2$ or 5 moves no pooled firing rate of the base rule by over 0.3 points, no blocked share by over 4.5 and no collateral median by over 7.2.

APPENDIX F REPRODUCIBILITY

The reference implementation is available at <https://github.com/NatanaelOliveira/knowledge-graph-ddos-article> under the MIT license. `make compile-check stix-ingest` checks the compiled count query on generated sessions and passes the exported bundles through the OASIS STIX 2.1 validator in strict mode, the reference TAXII 2.1 server (medallion 3.0.0), which returns them unchanged save the extension definition it fails to serve, and MISP’s importer (misp-stix 2026.9.16). The compiled query returned the exported origin and /24-pair counts in every window of two production days, 1 152 windows each, and the JA4 pair counts in the 288 windows a day of the endpoint where the WAF blocked no client. Elsewhere the export drops the clients the WAF partly blocked, which the compiled filter keeps. It ran in the operator’s log store, the artifact keeps only its counts, and the production evaluation reads the exports. The held-out day’s protocol was written down, with the hashes of its exports, before the day was read (`fresh_day_protocol.md`). It was committed 17 hours after the time it states, so no external timestamp attests that order. Seeds are fixed and Makefiles drive every stage, so each table and figure of generated traffic regenerates from one command. The production tables (`make production-tables waf-labels`) regenerate from per-window results kept with the operator’s data, and `make audit` recomputes about 260 of the printed numbers from the anonymized summaries in the repository. We intend to submit the artifacts to the Artifact Evaluation track.